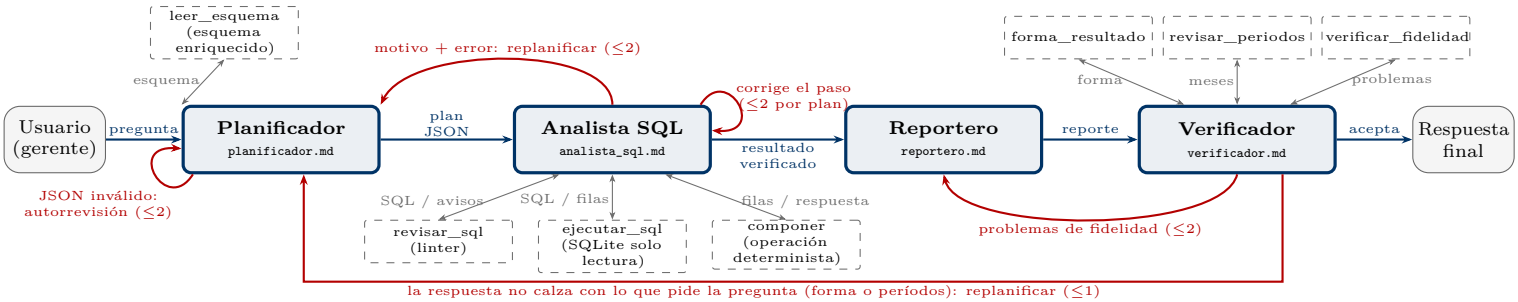
SQL4Business: Deliverable 3 — sistema de agentes

Generative Artificial Intelligence (580694), Universidad de Concepción
Aredhel Jiménez, Bryan Riquelme, Guido Salazar, Vicente Soñez

Código: github.com/aredhel-jmze/Asistente-SQL-4-Business | Video: [PENDIENTE: enlace del video]



Paradas: el Verificador acepta cuando no encuentra discrepancias o cuando se agotan los presupuestos (si un plan nuevo falla, entrega la respuesta anterior); sin respuesta previa, el Analista termina sin respuesta si el error persiste. Cajas azules: agentes (mismo modelo, prompt propio). Cajas grises: herramientas deterministas. Flechas rojas: ciclos, con su presupuesto.

Tarea, continuidad y modelo. La misma tarea, falla y criterio de salida correcta de los entregables anteriores. El sistema parte de la solución del D2: sus piezas deterministas (esquema enriquecido, SQL de solo lectura, composición, verificación de fidelidad) pasan a ser herramientas de los agentes, y el código del D2 no cambia. Los cuatro agentes usan el mismo modelo, `Qwen/Qwen2.5-Coder-3B-Instruct` (revisión `488639f`, NF4 de 4 bits, decodificación greedy, Colab T4), con un prompt propio por agente en `prompts/`.

Agentes y división del trabajo. El **Planificador** recibe la pregunta y el esquema y devuelve un plan JSON (pasos SQL y operación); si su JSON no es válido, revisa su propia salida. El **Analista SQL** ejecuta el plan con `revisar_sql`, `ejecutar_sql` y `componer`; solo llama al modelo cuando una herramienta falla, y entonces decide entre corregir el paso o devolver el plan con el motivo. El **Reportero** redacta una o dos oraciones desde el resultado verificado y las reescribe si el Verificador las rechaza. El **Verificador** lee solo la pregunta, declara qué tipo de respuesta pide y qué períodos nombra, y sus herramientas lo comparan con lo calculado: si no calza, devuelve el caso al Planificador; si el reporte no es fiel, al Reportero; si no, acepta. Se divide así porque son cuatro competencias distintas (entender la pregunta, interactuar con la base, redactar y juzgar) y porque quien juzga no debe ser quien produjo el trabajo.

Técnicas de prompting. Clasificadas según la taxonomía de Schulhoff et al. (2024). Todos los agentes usan *role prompting* y salida estructurada. *Planificador:* (1) **few-shot**, tres ejemplos resueltos sobre la misma base que no pertenecen a ningún conjunto de evaluación, contra la falla de devolver listados en vez del primero o de usar `strftime('%Q')`; (2) **descomposición** (*plan-and-solve*): antes del SQL declara qué debe contener la respuesta y cada período como rango de fechas, contra la falla de responder con una sola consulta una pregunta de varios valores. *Analista:* (3) **autocorrección guiada por herramientas** (*self-debugging*): el error de SQLite, del linter o de la composición vuelve al modelo, contra los errores de fechas y sintaxis (q09, q10 del D2). *Verificador:* (4) **verificación con lista de chequeo** (*self-verification*), contra respuestas válidas que no responden la pregunta (q03, q11, q09-p y q14-p del D2). *Reportero:* (5) **refinamiento con retroalimentación** (*self-refine*), contra cifras no respaldadas en el reporte (q13). Cada técnica se puede apagar por separado; la tabla mide el sistema sin cada una. Un test comprueba que los planes que fallaron en el D2 vuelven al agente que corresponde.

Evaluación. Las 15 preguntas del D1 (10 puntuales, 5 combinadas; in-sample) y 12 paráfrasis (5/7; externas). Mismas entradas y criterio para todas las configuraciones: *sql* (criterio del D1), *answer* (respuesta correcta), *e2e* (respuesta correcta y reporte fiel); *Llamadas*: promedio por pregunta.

Resultados. [PENDIENTE: completar con la tabla medida: agentes frente a D2 y baseline en combinadas y totales, y qué técnica pesa más en la ablación].

Caso de falla. [PENDIENTE: pregunta en la que el sistema completo falla, con su traza en `results/deliverable3/trazas/` y la explicación de por qué].

Límites. El Verificador depende de que un modelo de 3B clasifique bien la pregunta: una mala clasificación puede devolver un plan correcto y el nuevo puede ser peor. La revisión de períodos trabaja a nivel de mes y el linter solo reconoce un patrón de error silencioso. Un SQL válido con un predicado equivocado y un reporte fiel a ese resultado sigue sin detectarse. Cada pregunta cuesta más llamadas que en el D2. Ambos conjuntos son chicos y sintéticos.

Desviaciones declaradas. Nuevo linter `revisar_sql` (del caso de falla del D2); dos reglas al Reportero (sin pasos ni SQL; cambio porcentual una vez) y una a la verificación de fidelidad, compartida con el D2 (cambio porcentual repetido), que no altera veredictos del D2.

Reproducción. `notebooks/deliverable3_demo.ipynb` en una T4 de Colab: demo con los mensajes entre agentes (`scripts/run_agents.py`) y evaluación completa (`scripts/evaluate_d3.py`, unas 2 horas), que escribe `results/deliverable3/`, una traza por pregunta y esta tabla.

	Set oficial (15)			Paráfrasis (12)			
Configuración	sql	answer	e2e	sql	answer	e2e	**Llamadas**
Prompt directo (baseline D1)	?	?	–	?	?	–	?
Solución D2 (pipeline)	?	?	?	?	?	?	?
Sistema de agentes (D3)	?	?	?	?	?	?	?
sin ejemplos (few-shot)	?	?	?	?	?	?	?
sin descomposición explícita	?	?	?	?	?	?	?
sin autocorrección del Analista	?	?	?	?	?	?	?
sin verificación crítica	?	?	?	?	?	?	?
sin refinamiento del reporte	?	?	?	?	?	?	?